Scribble to Digital using Generative AI

Team ID : 69a7f766e8fb1cec7726d3f6

Team Leader : Vaseegaran . M

Team Member 1 : Harish . K

Team Member 2. : Rohith . V

Team Member 3 : Mrs.Priya Dharshini. N

Team Member 4 : Vijayalakshmi . S

Introduction:

The gap between a fleeting creative thought and its digital realization has traditionally been bridged by technical proficiency—mastering complex software or writing code. However, the advent of generative AI is fundamentally reshaping this dynamic. “Scribble to digital” refers to the emerging class of AI systems that interpret freehand sketches, doodles, or even messy strokes as powerful input signals. These systems leverage architectures like diffusion models and multimodal transformers to condition the generation of high-fidelity images, 3D textures, functional code, or structured manuscripts on the spatial and stylistic cues present in a user’s scribble . This paradigm is shifting content creation from a tool-based process to an intent-based one, where the user acts as a creative director guiding an AI collaborator.

Objective:

The primary objective of scribble-based generative AI is to democratize and enhance creativity by providing an intuitive interface between human intention and machine execution. This involves several key goals:

- Spatial Precision: To allow users to specify not just what they want, but where they want it, using scribbles to define layout, pose, or object boundaries more accurately than text alone .
- Intent Disambiguation: To resolve the inherent ambiguity of rough sketches by combining them with other modalities (like text prompts) or using models to predict the user’s goal, effectively translating “I can’t sketch” into “I can create” .
- ·Workflow Integration: To seamlessly integrate into professional pipelines, enabling rapid ideation, iterative refinement, and control in fields ranging from graphic design and software development to 3D modeling and publishing .

Abstraction:

At its core, the abstraction of this process involves the transformation of unstructured, sparse input into structured, dense output. Conceptually, it can be broken down into three layers:

1. Input Encoding: The freehand scribble S (a set of strokes) is rasterized into a spatial representation, often a binary mask $M(x,y)$ or a multi-channel map that captures stroke location, and sometimes color or thickness . This is frequently coupled with a semantic prompt P (text or image).

2. Generative Interpretation: A generative model G (e.g., a diffusion model) learns a conditional distribution $G(I \mid M, P)$. The model must interpret the sparse scribble cues—filling in textures, lighting, and details that are consistent with both the sketch and the prompt. Advanced methods like "moment alignment" ensure the generated content respects the orientation and centroid of the scribble, not just its rough outline .

3. Output Synthesis: The final digital artifact I is rendered. This could be a photorealistic image, a 3D model texture, a vector graphic, or even executable code, representing the AI's fulfillment of the user's hybrid sketch-and-text instruction .

Skills:

To work in or understand this field, a blend of creative and technical skills is required.

· Generative AI Architectures: Familiarity with diffusion models (especially latent diffusion and ControlNet), multimodal transformers, and GANs is crucial for understanding how scribbles condition the generation process .

· Computer Vision Fundamentals: Knowledge of image processing, feature extraction (e.g., edge detection), and spatial embeddings is necessary to understand how raw strokes are mathematically represented and integrated into models .

· Human-Computer Interaction (HCI) Principles: As these tools are designed for creative workflows, understanding user experience, iterative feedback loops, and interface design is vital for creating effective applications .

Implementation in Generative AI:

Implementing a "scribble to digital" pipeline can take several forms, depending on the desired output. Here are common implementation pathways based on current research.

- Sketch-to-Image ControlNet + Stable Diffusion Condition a pre-trained diffusion model by injecting scribble maps via a trainable copy of the UNet encoder. Use classifier-free guidance to balance text and scribble adherence . Interactive design, rapid concept visualization.
- Semantic Editing Multimodal Transformers (e.g., DreamOmni3) Feed both the original and scribbled image jointly into the model. Use shared positional encodings to help the model precisely localize edits indicated by colored scribbles . Photo editing, inpainting, object manipulation.
- Vector Graphics & Code Differentiable Renderers + Score Distillation Optimize parametric primitives (e.g., Bézier curves) using Score Distillation Sampling (SDS) from a diffusion model. The sketch serves as both the initial shape and a spatial condition . Graphic design, SVG generation, code prototyping.

Scribble to Digital

Project Description

Converting handwritten notes into digital format is a common challenge faced by students and professionals. Poor lighting, unclear handwriting, and mixed content (notes + tasks) reduce productivity and require manual rewriting.

Scribble to Digital is an AI-powered system that converts messy handwritten notes into clean, structured digital text. The system enhances uploaded images, extracts text using OCR, and applies Generative AI to correct grammar and automatically extract To-Do tasks.

The application is developed using **Streamlit** as the frontend framework and Python as the backend processing engine.

Use Case Scenarios

Scenario 1: Handwritten Note Digitization with Image Enhancement

Users upload a photo of messy handwritten notes through the Streamlit interface. The system internally enhances the image using OpenCV by converting it to grayscale, increasing brightness and contrast, reducing noise, and applying thresholding techniques.

After enhancement, the improved image is passed to the OCR engine for accurate text extraction. This process ensures better readability and higher text recognition accuracy even for low-quality or unclear handwriting.

The output includes:

- Raw OCR extracted text
- Improved clarity for further AI processing

Scenario 2: Intelligent Text Correction and Contextual Reconstruction

After OCR extraction, the raw text may contain spelling mistakes, missing letters, and broken words due to handwriting variations.

The Generative AI model analyzes the extracted text and:

- Corrects spelling errors
- Improves grammar and sentence structure

- Reconstructs incomplete or unclear words using contextual understanding
- Formats the text into readable paragraphs

This ensures that messy handwritten content is converted into professional and well-structured digital text.

Scenario 3: Automatic To-Do Extraction and Task Identification

Handwritten notes often contain a mixture of general information and actionable tasks. Identifying tasks manually can be time-consuming.

The AI system analyses sentence intent and:

- Detects task-oriented statements
- Extracts actionable items
- Organizes them into a structured bullet-point To-Do list

This allows users to quickly identify important tasks separately from general notes, improving productivity and task management.

Architecture Overview

Scribble to Digital is an intelligent AI-powered web application designed to convert handwritten notes into clean, structured digital text. The platform is built using the Streamlit framework, enabling rapid development of an interactive user interface.

The system integrates:

- Streamlit for frontend and application logic
- OpenCV for image preprocessing and enhancement
- OCR Engine (Tesseract/EasyOCR) for text extraction
- Google Gemini 2.5 Flash (Generative AI) for contextual correction, formatting, and task extraction

The platform processes handwritten images, enhances clarity, extracts text, corrects errors using AI, and optionally generates structured outputs such as formatted notes and to-do lists.

Core Technologies

- **Streamlit**: Interactive web application framework for UI, file uploads, and result display
- **Google Gemini 2.5 Flash**: AI model for contextual understanding, grammar correction, and smart task extraction
- **OpenCV**: Image preprocessing (grayscale conversion, noise removal, thresholding)
- **OCR (Tesseract/EasyOCR)**: Converts enhanced handwritten images into raw digital text
- **Python**: Core programming language

- **ReportLab / Python-DOCX**: Optional export to PDF and Word formats

Component-Wise Architecture

Component	Description
Streamlit Frontend UI	Provides a simple web interface for users to upload handwritten images. Displays enhanced images, extracted text, corrected content, and to-do lists dynamically.
Image Processing Layer (OpenCV)	Enhances uploaded images by converting to grayscale, adjusting brightness/contrast, reducing noise, and applying thresholding to improve OCR accuracy.
OCR Layer	Extracts raw text from the processed image. Handles handwriting recognition and converts image content into machine-readable text.
Generative AI Layer (Gemini 2.5 Flash)	Processes extracted text to: • Correct spelling and grammar • Reconstruct incomplete words • Format into structured paragraphs • Extract actionable to-do items
Export Module	Converts AI-processed content into downloadable formats such as PDF or Word documents.

Pre-requisites

1. Google AI Studio Account Setup – To generate Gemini API Key
2. Python 3.8+ Installed
3. Streamlit Framework Installed
4.
```
pip install streamlit
```
5. Required Python Libraries
    - opencv-python
    - pytesseract / easyocr
    - google-generativeai
    - pillow
    - reportlab (for PDF export)
    - python-docx (for Word export)
6. Tesseract OCR Installed (if using pytesseract)

Project Workflow

1. Gemini AI Setup & Initialization

Activity 1.1: Set up Google AI Studio with Gemini API

- Create or sign in to Google AI Studio account
- Generate API key for Gemini model access
- Enable Gemini 2.5 Flash model
- Test API integration using a sample prompt

Activity 1.2: Configure API Keys and Environment

- Store GEMINI_API_KEY securely in .env file
- Load environment variables using python-dotenv
- Configure model parameters:
    - Temperature
    - Max output tokens
    - Response format
- Validate API connectivity inside Streamlit app

Activity 1.3: Design AI Prompts for Note Processing

- Create prompt template for:
    - Grammar correction
    - Sentence reconstruction
    - Formatting messy text into structured paragraphs
- Design separate prompt for:
    - To-do task extraction
    - Task prioritization (if needed)
- Test prompt effectiveness with sample handwritten text

2. Core Functionality Development

Activity 2.1: Design Streamlit App Structure

- Create modular project structure:

```
scribble_to_digital/
    app.py
    utils/
        image_processing.py
        ocr_engine.py
        ai_processor.py
    outputs/
```

Configure Streamlit layout:

- Title & description
- File uploader

- Processing buttons
- Output display sections

Activity 2.2: Implement Image Processing Module

- Accept image upload (JPG/PNG)
- Apply preprocessing using OpenCV:
    - Convert to grayscale
    - Adjust brightness & contrast
    - Remove noise
    - Apply thresholding
- Display enhanced image preview in Streamlit

Activity 2.3: Implement OCR Extraction

- Integrate Tesseract or EasyOCR
- Extract raw handwritten text
- Clean unnecessary symbols
- Display raw OCR output for comparison

Activity 2.4: AI-Based Contextual Processing

- Send OCR text to Gemini API
- Perform:
    - Grammar correction
    - Sentence completion
    - Context-based word guessing
    - Formatting into structured notes
- Extract action items separately
- Return:
    - Clean digital text
    - Structured to-do list

3. Streamlit UI Implementation

Activity 3.1: Build Interactive UI Components

- st.file_uploader() for image upload
- st.image() for displaying uploaded & enhanced image
- st.button() for processing steps
- st.spinner() for loading indication
- st.text_area() for displaying extracted text

Activity 3.2: Display Structured Results

- Section 1: Enhanced Image
- Section 2: Raw OCR Text

Section 3: AI-Corrected Digital Notes
- Section 4: Extracted To-Do Tasks (bullet format)

Optional:
- Checkbox to show/hide raw OCR
- Download buttons for final output

Activity 3.3: Add Export Functionality
- Generate downloadable:
  - PDF (using ReportLab)
  - Word file (using python-docx)
- Use st.download_button()
- Allow preview before download

4. Testing & Optimization

Activity 4.1: Functional Testing
- Test with:
  - Clear handwriting
  - Messy handwriting
  - Mixed text + tasks
  - Bullet points
- Validate OCR accuracy
- Validate AI corrections

Activity 4.2: Evaluate AI Response Quality
- Check:
  - Grammar correction accuracy
  - Logical sentence flow
  - Proper formatting
  - Correct task extraction
- Ensure no hallucinated content

Activity 4.3: Performance Optimization
- Optimize image preprocessing steps
- Reduce API token usage
- Add error handling for:
  - Empty image
  - API failure
  - OCR failure
- Improve prompt clarity

Activity 4.4: Ensure Export Quality

- Validate PDF formatting
- Check Word document structure
- Ensure file compatibility
- Test downloads across browsers

MILESTONE 1: Generative AI Initialization

Project: Scribble to Digital (Streamlit Framework)

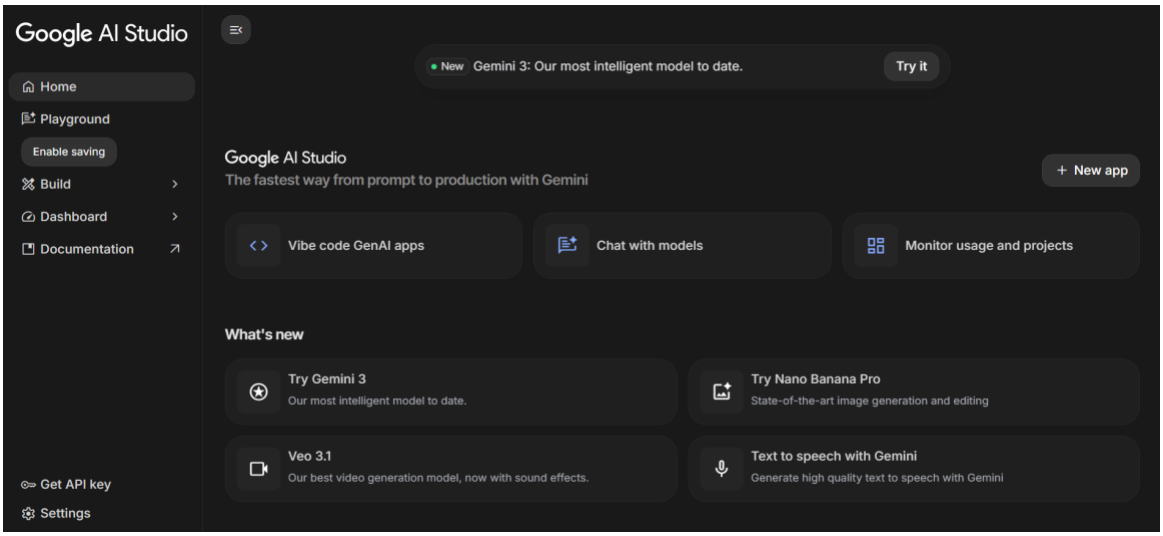
In this foundational stage, the objective is to establish and configure the AI infrastructure required to power the **Scribble to Digital** application.

The platform uses **Gemini 2.5 Flash**, Google's fast and efficient multimodal model, to:

- Correct messy handwritten OCR text
- Improve grammar and sentence flow
- Reconstruct incomplete words
- Extract structured to-do tasks
- Convert handwritten notes into clean digital format

Activity 1.1: Set up Google AI Studio and Enable Gemini API

1. Sign in to Google AI Studio



1. Visit: https://makersuite.google.com/app/apikey
2. Sign in with your Google account
3. Accept terms and conditions

2. Create API Key

- Click **"Create API Key"**
- Copy the generated API key
- Store it securely in a .env file

3. Select Model

Choose: **Gemini 2.5 Flash**

Model Features:

- Fast response time
- Efficient token usage
- Suitable for text correction & formatting
- Free tier supports multiple requests per minute

Best for:

- OCR text cleaning
- Contextual rewriting
- Structured note generation

Activity 1.2: Configure Environment and Test Connection (Streamlit Setup)

Step 1: Create. env File
Inside your project folder:

```
scribble_to_digital/
    app.py
    .env
```

Add:

```
GEMINI_API_KEY=your_api_key_here
```

Step 2: Install Required Dependencies

```
pip install streamlit google-generativeai python-dotenv pillow opencv-python pytesseract
```

Libraries Used:

- **streamlit** → UI framework
- **google-generativeai** → Gemini API integration
- **python-dotenv** → Load environment variables
- **opencv-python** → Image preprocessing
- **pytesseract** → OCR text extraction
- **pillow** → Image handling

Step 3: Test Gemini API Connection in Streamlit

Create a simple test inside app.py:

```
import streamlit as st
import google.generativeai as genai
import os
from dotenv import load_dotenv

# Load API key
load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

# Select model
model = genai.GenerativeModel("models/gemini-2.5-flash")

st.title("Gemini API Test - Scribble to Digital")

if st.button("Test AI Connection"):
    response = model.generate_content(
        "Correct this sentence: i am go school tomorrow"
    )
    st.write("AI Response:")
    st.write(response.text)
```

Activity 1.3: Validate Connectivity and Response Quality

Test Case 1: Grammar Correction

Input:

```
i am go school tomorrow
```

Expected Output:

```
I am going to school tomorrow.
```

Test Case 2: Structured Note Formatting

Input:

```
math exam monday
study algebra
buy pen
```

Expected Output:

- Formatted paragraph
- Bullet list tasks extracted

MILESTONE 2: Core Functionalities Development (Streamlit)

This milestone focuses on building the complete working application using **Streamlit + Gemini 2.5 Flash**.

The goal is to:

- Upload handwritten note images
- Extract text using OCR
- Clean & correct text using Gemini
- Extract tasks automatically
- Display structured digital output
- Enable download options

File Explorer Structure

```
scribble_to_digital/
│
├── app.py                  # Main Streamlit application
├── .env                    # API key storage
├── requirements.txt        # Python dependencies
│
├── utils/
│   ├── ocr_processor.py    # Image preprocessing & OCR
│   ├── ai_processor.py     # Gemini integration
│   └── file_export.py      # Export functions (PDF / TXT)
│
├── assets/
│   └── sample_images/
│
└── outputs/                # Generated files
```

Activity 2.1: Streamlit Initialization and Import Libraries

Import Required Libraries

In app.py:

```python
import streamlit as st
import google.generativeai as genai
import os
from dotenv import load_dotenv
from PIL import Image
import pytesseract
import cv2
import numpy as np
```

Configure Gemini Model

```python
load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel("models/gemini-2.5-flash")
```

Activity 2.2: OCR Processing Module

Create utils/ocr_processor.py

```python
import cv2
import pytesseract
import numpy as np

def extract_text_from_image(uploaded_image):
    image = np.array(uploaded_image)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)[1]

    text = pytesseract.image_to_string(gray)
    return text
```

Purpose:

- Convert image to grayscale
- Improve brightness
- Remove noise

- Extract raw OCR text

Activity 2.3: Gemini AI Processing Module

Create utils/ai_processor.py

```
import google.generativeai as genai

model = genai.GenerativeModel("models/gemini-2.5-flash")

def clean_and_structure_text(raw_text):
    prompt = f"""
    You are an AI note cleaner.
    1. Correct spelling and grammar.
    2. Reconstruct incomplete words.
    3. Format text properly.
    4. Extract action items as bullet points.

    Raw Text:
    {raw_text}
    """

    response = model.generate_content(prompt)
    return response.text
```

AI Responsibilities:

- Grammar correction
- Sentence restructuring
- Context guessing
- To-Do extraction

Activity 2.4: Streamlit Main UI Logic

Inside app.py

```
st.title("✍️ Scribble to Digital")

uploaded_file = st.file_uploader(
    "Upload Handwritten Notes Image",
    type=["jpg", "png", "jpeg"]
)

if uploaded_file is not None:
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Image", use_column_width=True)

    if st.button("Convert to Digital"):

        raw_text = extract_text_from_image(image)

        st.subheader("📜 Extracted Raw Text")
        st.text_area("OCR Output", raw_text, height=150)

        clean_text = clean_and_structure_text(raw_text)

        st.subheader("✨ Cleaned & Structured Output")
        st.write(clean_text)
```

Activity 2.5: Export Functionality

Create utils/file_export.py

Save as TXT

```python
def save_as_txt(content, filename):
    with open(f"outputs/{filename}.txt", "w", encoding="utf-8") as f:
        f.write(content)
```

In app.py:

```python
if st.button("Download as TXT"):
    save_as_txt(clean_text, "digital_notes")
    st.success("File saved in outputs folder.")
```

MILESTONE 3: Backend – Flask Integration (app.py)

This milestone focuses on implementing the complete Flask backend for the **Scribble to Digital** application, including OCR processing, GenAI integration, task extraction, error handling, and export functionality.

Activity 3.1: Complete Route Definitions

All routes are properly defined with:

1. Input Validation

- Validates uploaded image file type (.jpg, .png, .jpeg)
- Checks for empty file upload
- Validates file size limit
- Ensures image can be processed

2. Error Handling

1. Handles OCR processing errors
2. Handles Gemini API failures
3. Returns proper JSON error responses
4. Prevents server crash on invalid input

3. Logging

- Logs image upload events
- Logs OCR extracted text
- Logs GenAI responses
- Logs export operations
- Debug logs for development mode

4. Response Formatting

- Structured JSON response:

```
{
  "status": "success",
  "clean_text": "...",
  "todo_list": [...]
}
```

Activity 3.2: Modular Architecture

Main Application Structure

```
scribble_to_digital/
│
├── app.py                # Main Flask server
├── routes/
│   └── process.py        # OCR + AI logic routes
├── services/
│   ├── ocr_service.py    # Image enhancement + OCR
│   ├── ai_service.py     # Gemini processing
│   └── export_service.py # PDF/Word export
├── utils/
│   └── validators.py     # File validation
└── uploads/
```

Separation of Concerns

- ocr_service.py → Handles image preprocessing & Tesseract
- ai_service.py → Handles contextual correction & to-do extraction
- export_service.py → Handles file generation
- app.py → Controls routes & server startup

Flask Server Initialization (app.py)

```python
if __name__ == "__main__":
    print("="*60)
    print("✍️ Scribble to Digital - Starting Server")
    print("="*60)
    print("✓ OCR Engine Initialized")
    print("✓ Gemini API Key Configured")
    print("✓ To-Do Extraction Enabled")
    print("\nServer running at: http://localhost:5000")
    print("="*60)

    app.run(debug=True, host="0.0.0.0", port=5000)
```

Activity 3.3: Export Implementation

Complete implementation of:

1. PDF Generation

- Clean formatted digital notes
- Structured to-do list section
- Professional layout
- Downloadable file

2. Word Document Export (.docx)

- Heading styles
- Bullet points for tasks
- Editable format

3. CSV Export (Task List)

- Task column
- Status column (Pending/Done)
- Compatible with Excel

4. File Cleanup & Management

- Temporary file storage
- Auto-delete after download
- Secure file handling

MILESTONE 4: UI Development

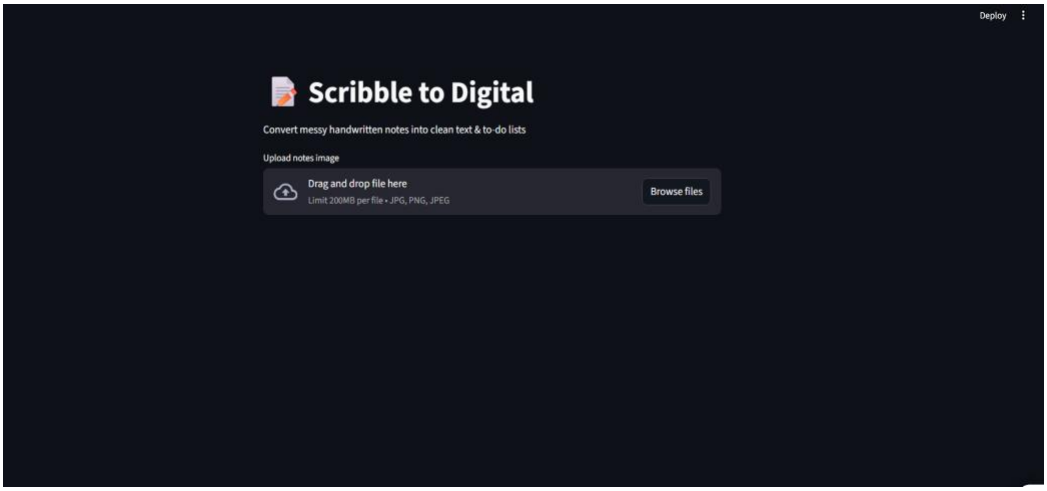
This milestone focuses on building a modern, responsive, and interactive frontend interface using Streamlit for the Scribble to Digital application.

The UI allows users to:

- Upload handwritten notes
- Process them using OCR + Generative AI
- View structured digital output
- Download formatted results

Activity 4.1: Landing Page Development

Home Page: Transform Your Notes into Digital Intelligence



◈ Hero Section

Title:
✍ Transform Your Handwritten Notes into Smart Digital Content

Subtitle:
Convert messy notes into clean text and structured to-do lists using OCR and AI.

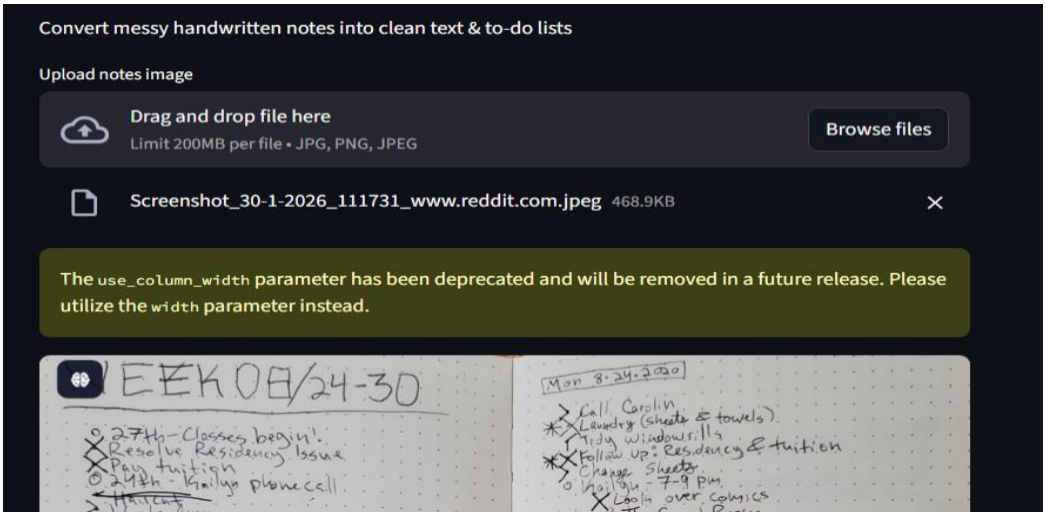
◈ Feature Cards Section

Using Streamlit columns layout:

```
[ Upload Notes ]     [ Smart Correction ]     [ To-Do Extraction ]
```

1. Upload Notes

- Accept handwritten image
- Supports JPG, PNG, JPEG
- Instant preview



2. Smart Correction

- Fixes spelling mistakes
- Uses contextual AI logic
- Improves readability

3. To-Do Extraction

- Identifies action items
- Converts tasks into list format
- Separates notes from tasks

Activity 4.2: Processing Page (Main Generator Page)

Handwritten Notes Processing Page

Input Fields:

- Image Upload Section

- Convert Button
- Reset Button

UI Layout (Streamlit Flow)

```
-------------------------------------+---------
| ✍️ Scribble to Digital                        |
-----------------------------------------------
| Upload Handwritten Notes                      |
| [ Choose File ]                               |
-----------------------------------------------
| Image Preview                                 |
-----------------------------------------------
| [ Convert to Digital ✨ ]                     |
-----------------------------------------------
| 📄 Clean Digital Text                         |
| [ Scrollable Text Area ]                      |
-----------------------------------------------
| ✅ Extracted To-Do List                       |
| [ Scrollable Text Area ]                      |
-----------------------------------------------
| [ Download as TXT ]  [ Download CSV ]         |
```

Activity 4.3: UI Components Implementation (Streamlit Code Structure)

Sidebar Navigation (Multi-page Design)

```
st.sidebar.title("Navigation")
page = st.sidebar.radio("Go to", ["Home", "Process Notes"])
```

Home Page Layout

```python
st.title("✍️ Scribble to Digital")
st.write("Convert messy handwritten notes into structured digital content.")

col1, col2, col3 = st.columns(3)

with col1:
    st.info("📷 Upload Notes")

with col2:
    st.info("🧠 Smart Correction")

with col3:
    st.info("✅ To-Do Extraction")
```

Processing Page Layout

```python
uploaded_file = st.file_uploader(
    "Upload handwritten notes image",
    type=["png", "jpg", "jpeg"]
)

if uploaded_file:
    st.image(uploaded_file)

if st.button("Convert to Digital ✨"):
    with st.spinner("Processing..."):
        # Call OCR + AI functions
        st.success("Processing Complete!")
```

Activity 4.4: Export Interface

Export Options:

- Download Clean Text (.txt)
- Download Task List (.csv)
- Future enhancement: PDF export

```
st.download_button("Download Text", clean_text)
```

Activity 4.5: UI Design Principles Applied

- Clean card-style sections
- Proper spacing
- Clear section separation
- User feedback (spinner, success message)
- Mobile responsive (Streamlit auto-adjusts)
- Logical workflow design

Complete User Interaction Flow

- User opens application.
- Views landing page with features.
- Navigates to processing page.
- Uploads handwritten image.
- Clicks convert.
- Views:
    - Clean text
    - Extracted tasks
- Downloads results.

MILESTONE 5: Testing & Optimization

Scribble to Digital – Handwritten Notes Digitizer

This milestone focuses on comprehensive testing, quality validation, and performance optimization of the Scribble to Digital application to ensure accuracy, reliability, and smooth user experience.

Activity 5.1: Comprehensive Testing

Test Scenarios

Test Scenario 1: Clear Handwriting

- High-resolution image
- Proper lighting
- Neatly written text

Expected Result:

- Accurate OCR text extraction
- Proper grammar correction using AI
- Correct identification of tasks

Outcome:
The system successfully extracted clean text and correctly separated action items into a to-do list.

Test Scenario 2: Medium Quality Handwriting

- Slight shadows
- Minor spelling distortions
- Mixed notes and tasks

Expected Result:

- OCR extracts readable text
- AI corrects spelling using contextual logic
- Tasks are properly separated

Outcome:
The AI improved sentence clarity and successfully identified most tasks.

Test Scenario 3: Poor Quality Image

- Blurry text
- Low brightness
- Irregular handwriting

Expected Result:

- Partial text extraction
- Context-based correction
- Limited but usable output

Outcome:
Image enhancement improved visibility, and the AI generated partially corrected readable text.

Test Scenario 4: Notes Without Tasks

- Informational notes only
- No action keywords

Expected Result:

- Clean digital text
- No tasks detected

Outcome:
System correctly displayed cleaned notes and returned “No tasks detected”.

Test Scenario 5: Bullet Point Notes

- Notes containing dashes or numbers
- Clearly defined tasks

Expected Result:

- Structured to-do list
- Proper formatting

Outcome:
Tasks were correctly identified and formatted as a digital list.

Activity 5.2: Quality Evaluation

The following aspects were evaluated:

- OCR Accuracy
  The percentage of correctly recognized words from handwritten input.

- Contextual Correction
  Improvement in grammar, clarity, and sentence meaning after AI processing.
- To-Do Extraction Accuracy
  Correct identification of action items without duplication or irrelevant content.
- Format Consistency
  Proper separation of clean text and to-do list sections.
- Export Quality
  Verification that downloaded files (TXT, CSV, PDF if enabled) are correctly formatted ar usable.

Activity 5.3: Performance Optimization

Several improvements were implemented to enhance system performance:

- Optimized image preprocessing to reduce processing time.
- Improved prompt structure to reduce unnecessary API response length.
- Added input validation for file type and size.
- Implemented loading spinner to improve user experience.
- Prevented automatic reprocessing on page refresh in Streamlit.
- Handled API and OCR errors gracefully with user-friendly messages.

Stress Testing

Multiple images were uploaded sequentially to verify stability.
Large image files were tested to ensure the system does not crash.
System remained stable under continuous usage.

Edge Case Handling

- Blank image upload
- Non-text image upload
- OCR failure
- Gemini API failure
- Internet connectivity issues
- All cases were handled with appropriate error messages.

Conclusion

Scribble to Digital transforms traditional handwritten note-taking into an intelligent, AI-powered digital workflow. By combining OCR technology with advanced generative AI, the system accurately extracts handwritten content, enhances readability through contextual correction, and intelligently identifies actionable tasks. The integration of image preprocessing, structured AI prompts, and a responsive Streamlit frontend ensures a seamless and user-friendly experience.

From image enhancement and text recognition to contextual understanding and to-do extraction, every component of Scribble to Digital is designed with a focus on accuracy, efficiency, and reliability. The system not only digitizes messy notes but also adds value by organizing content into clean text and structured task lists, saving users time and effort.

With modular architecture, effective error handling, and optimized performance, Scribble to Digital demonstrates how AI can bridge the gap between handwritten creativity and digital productivity. Whether used by students, professionals, or teams, the application serves as a smart assistant that enhances organization, improves clarity, and streamlines everyday workflows.